AI Image Generation using Diffusion Model

Lian Choi

CID: 01704485

Supervised by

Dr C. Ling

Second Marker

Prof. A. Another

A Thesis submitted in fulfilment of the requirements for the degree of

Bachelor of Engineering in Electronic and Information Engineering

Department of Electrical and Electronic Engineering
Imperial College London

2024

Abstract

Denoising Diffusion Probabilistic Models (DDPMs) have rapidly become one of the leading families of generative models, offering a stability of training that Generative Adversarial Networks (GANs) have historically struggled to achieve. Within a DDPM the diffusion process is fixed and the only learned component is the network that predicts the noise added at each step, so the choice of that network governs the quality of the images produced. This thesis asks whether combining residual learning with the UNet backbone conventionally used for that purpose yields a measurable improvement.

A residual UNet (ResUNet) was implemented in PyTorch, in which every encoder and decoder stage is constructed from residual blocks rather than plain convolutional stacks, and trained directly on the standard DDPM objective. It is assessed against two baselines under matched conditions. The first removes the identity shortcut from each block and is otherwise identical, so that the two networks have exactly the same parameter count and the comparison isolates residual learning as the single variable. The second is the denoising architecture of the original DDPM, with four resolution stages and self-attention, reduced in width so that its parameter count agrees with the proposed network to within ten per cent. All three were trained on CIFAR-10 at 32×32 and on CelebA at 64×64, sharing the diffusion schedule, the optimiser, the number of epochs and the random seed, and sampling from the same initial noise.

The three networks were compared on restoration accuracy, measured by structural similarity and peak signal-to-noise ratio against held-out images at six noise levels; on distribution-level quality, measured by the Fréchet Inception Distance and the Inception Score over two thousand generated samples; and on a nearest-neighbour check against the training set that found no evidence of memorisation on either dataset.

Every configuration was trained under three random seeds. The residual shortcut lowers the first-epoch loss in all six paired runs, by five to eleven per cent on average, but that advantage disappears by the fifth epoch, and at convergence the restoration measures cannot separate the two networks: the training loss leans towards the plain variant and structural similarity towards the proposed one, both by less than half of one per cent. Generative quality separates them, on the deeper configuration only. On CelebA the proposed network leads in every seed, by a mean of 63.31 FID against a seed-to-seed standard deviation of about 23; on CIFAR-10 the paired difference is -3.46 ± 13.78 with the sign changing between seeds. Two networks that denoise equally well are therefore shown to generate images of very different quality, a discrepancy no restoration measure anticipated. The reference architecture remains the strongest overall, leading every restoration measure and CIFAR-10 FID at roughly 2.3 times the training cost, and is matched only on CelebA FID by the proposed network.

The hypothesis accordingly receives partial and conditional support: residual learning does not improve generated image quality at two resolution stages, where it makes no detectable difference, but at three stages it leads in every seed by a margin larger than the observed spread. Negative Log-Likelihood was excluded on the grounds that this implementation fixes the reverse-process variances rather than learning them, and the comparison against Variational Autoencoder and GAN baselines was not carried out.

Contents

1 Project Specification

1.1 Introduction

This research project is centered on examining the capabilities of Denoising Diffusion Probabilistic Model (DDPM) in the field of image generation. The focus of this study is the generation of natural images, evaluated on two datasets of differing resolution. The first is CIFAR-10, the standard benchmark against which diffusion models are reported, comprising 32×32 RGB images drawn from ten object classes; retaining it allows the results obtained here to be placed alongside the published literature. The second is CelebA, a large-scale collection of aligned face photographs, from which a centre crop is taken and resized to 64×64. The higher resolution is included because architectural differences between denoising networks are difficult to observe at 32×32, where the spatial extent is exhausted after only two downsampling stages; at 64×64 a third stage becomes available and the contribution of network depth and of the skip connections is visible both in the metrics and in the generated samples. So that the two settings remain comparable in training cost, a 50,000-image subset of CelebA is used, matching the size of the CIFAR-10 training split. The project aims to not only develop and train a DDPM model but also to enrich the research with comprehensive analytical work, simulations, and the development of dedicated software tools. This approach intends to merge theoretical insights with practical applications, contributing significantly to the understanding and advancement of generative models in digital image processing.

In this research project, I will showcase the generative capabilities of diffusion models by comparing them to established competitors in the field of image generation: Generative Adversarial Network (GANs) and Variational Auto Encoder (VAEs). This study aims to provide a thorough comparative analysis, highlighting the strengths and limitations of each model type. Additionally, the project will explore an alternative approach by combining residual learning with the UNet backbone that is conventionally adopted for the denoising network, giving a residual UNet (ResUNet) in which each encoder and decoder stage is constructed from residual blocks rather than plain convolutional stacks. This investigation into different architectural frameworks is expected to yield insightful results regarding efficiency, effectiveness, and the quality of image generation, thus contributing to the broader discourse on machine learning architectures in generative applications.

1.1.1 Software

This project involves the implementation of a Denoising Diffusion Probabilistic Model (DDPM) with an emphasis on the denoising process, for which a residual UNet (ResUNet) serves as the noise-prediction network. Training follows the standard DDPM objective: a timestep is drawn uniformly for each example in a batch, the corresponding noisy image is formed in closed form from the cumulative noise schedule, and the network is optimised to predict the Gaussian noise that was added. The quantity minimised is therefore a noise-prediction error, not a reconstruction error. A key component of the framework is the data handling routine, which caches CIFAR-10 and CelebA as tensors at the required resolution so that an identical training loop applies to either dataset without modification. Dedicated classes are provided for training, sampling and evaluation, and the diffusion process is implemented separately from the denoising network so that the architecture under test can be exchanged without altering the generative model itself. This separation is what makes the controlled comparison of Section 1.1.2 possible.

1.1.2 Simulation

The experimental programme holds the diffusion framework fixed and varies only the denoising network, so that any difference in performance can be attributed to the architecture rather than to the generative model. Three networks are trained on each of the two datasets, giving six runs in total.

- **ResUNet** is the proposed network, in which every encoder and decoder stage is built from residual blocks.
- **Plain UNet** is identical in every respect except that the identity shortcut is removed from each block. A shortcut adds no parameters, so the two networks have exactly the same parameter count and computational cost; this comparison therefore isolates residual learning as the single variable.
- **DDPM UNet** is the denoising architecture of Ho et al., with four resolution stages, self-attention at 16×16 and a skip connection taken from every residual block. Its base channel width is reduced so that its parameter count falls within a few per cent of the proposed network. Without that adjustment the published configuration would be an order of magnitude larger, and any difference in quality would reflect capacity rather than design.

All six runs share the number of diffusion steps, the noise schedule, the training objective, the optimiser and its learning rate, the number of epochs, the data augmentation and the random seed; sampling for every model begins from the same initial noise. Evaluation is carried out on three levels. The restoration diagnostics are computed over a thousand held-out test images at six separate noise levels; the distribution-level metrics require a far larger sample and are computed from two thousand images drawn through the complete reverse process for each model; and the memorisation check compares every generated sample against the entire training set. Analysing the models under these matched conditions is what allows patterns and variances in the diffusion process to be attributed to the architecture, and provides the basis on which the proposed network is judged.

1.1.3 Analytical Work

The analysis reported in Chapter 3 follows the design set out above. Every configuration is trained under identical conditions and evaluated by the same procedure, so that the measurements can be compared directly; the decisions taken during development, and the corrections made when a measurement proved to be reporting something other than what was intended, are recorded alongside the results rather than omitted from them. Two such corrections proved substantive enough to affect the conclusions and are documented in place: the numerical instability of the reverse process, and the inability of the restoration measures to anticipate generative quality.

No single number captures the performance of a generative model, and the measurements reported here fall into three groups that answer different questions. The first group asks how faithfully a corrupted image is restored, and comprises the Structural Similarity Index Measure (SSIM) and the Peak Signal-to-Noise Ratio (PSNR). The second asks how closely the distribution of generated images resembles the distribution of real ones, and comprises the Fréchet Inception Distance (FID) and the Inception Score (IS). The third is not a quality measure at all but a validity check: a nearest-neighbour comparison against the training set, which establishes that the model is generating rather than reproducing.

- **Structural Similarity Index Measure** compares two images in terms of luminance, contrast and structural information. It requires a matched reference image, and is therefore applied to the reconstruction of held-out test images rather than to unconditional samples: a test image is corrupted to a chosen timestep and recovered in a single step, and the recovered image is compared with the original. A higher value indicates a more faithful restoration.

- **Peak Signal-to-Noise Ratio** is computed on the same pairs and reports the mean squared error on a logarithmic scale, in decibels. It is reported alongside SSIM because the two quantify different things: SSIM responds to structural agreement, PSNR to pixel-wise error. Where the two disagree, the disagreement is itself informative, since it indicates that one network is more accurate in absolute terms while the other preserves structure better.

- **Fréchet Inception Distance** quantifies the distance between the feature vectors of real images and generated images calculated by a pre-trained Inception network. Unlike the two measures above it needs no reference image, so it applies to samples drawn from noise and is therefore the primary measure of generative quality in this work. A lower FID indicates a closer match between the generated and real distributions.

- **Inception Score** evaluates the clarity and diversity of generated images from the same samples. A higher IS suggests that the model produces distinct and confidently classified images, although the measure is known to reward confident classification even when the images are not realistic, and it is therefore read alongside FID rather than on its own.

- **Nearest-neighbour distance** locates, for every generated sample, the closest image in the training set under a pixel-space L_2 metric. A distance on its own cannot be interpreted, so the same measurement is taken for held-out real images that the model never saw; if the generated samples are no closer to the training set than genuine unseen images are, there is no evidence that the model is reproducing its training data.

Negative Log-Likelihood was considered and deliberately excluded. Obtaining it for a diffusion model requires evaluating the full variational bound rather than the simplified training objective, and the bound is dominated by the reverse-process variances. The implementation used here follows the original formulation in fixing those variances to the posterior values rather than learning them, so the resulting likelihood would report a property of that fixed choice more than a property of the denoising network under test. Reporting it would add a number without adding evidence about the question this project asks.

Taken together the three groups support different claims. The restoration measures compare the networks as denoisers, which is what they are trained to be; the distribution measures compare them as generators, which is what they are ultimately for; and the memorisation check establishes that the second claim is meaningful. Reporting all three guards against the failure mode of selecting whichever single metric happens to favour the proposed architecture.

2 Background

2.1 Review of Literature

Diffusion probabilistic models have emerged as a powerful class of generative models that have significantly advanced the field of image generation. These models operate by gradually transforming a distribution of random noise into a distribution of structured images over many iterative steps, effectively reversing the diffusion process. This methodology allows for the generation of highly detailed and coherent images across a range of domains and applications.

One of the most notable achievements of diffusion probabilistic models is their ability to produce images that rival the quality of those generated by other leading techniques, such as Generative Adversarial Networks (GANs). Studies have shown that diffusion models can achieve superior fidelity and diversity in generated images, often demonstrating fewer artifacts than those commonly seen in GAN-produced images.

In terms of specific accomplishments, diffusion models have excelled in tasks such as high-resolution image synthesis, text-to-image generation, and image-to-image translation. For instance, they have been successfully applied to create photorealistic images of landscapes and portraits from textual descriptions, showcasing their versatility and robustness. Additionally, these models have proven capable of handling complex conditioning information, making them particularly useful for tasks requiring detailed customization of the generated content.

The theoretical underpinnings of diffusion models also contribute to their appeal. Unlike GANs, which can be challenging to train due to issues like mode collapse, diffusion models offer a more stable training process. This stability stems from their foundation in well-understood stochastic differential equations, providing a clear pathway for systematic improvements and adaptations.

Overall, diffusion probabilistic models represent a significant step forward in the field of image generation, combining high-quality output with robust and stable training methodologies. Their continuing development promises further enhancements in generating detailed, diverse, and customizable visual content, making them a central focus of current research in artificial intelligence and computer vision.

The ResNet architecture, short for Residual Network, is a groundbreaking innovation in the field of deep learning that was introduced by Kaiming He et al., titled "Deep Residual Learning for Image Recognition," [1]. This architecture has since been widely adopted for various applications across computer vision tasks due to its powerful and efficient design.

ResNet's primary innovation lies in its use of residual blocks (Figure 2.1). These blocks allow the network to skip one or more layers through the use of shortcut connections, or skip connections, that perform identity mapping. By adding the output from an earlier layer to a later

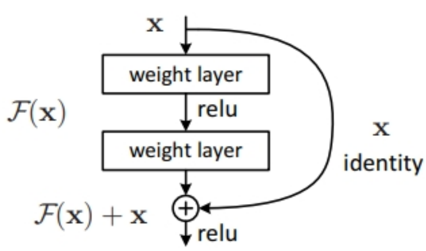


Figure 2.1: Residual Learning: a building block [1]

layer, these connections help to mitigate the vanishing gradient problem that plagues deep neural networks. As a result, ResNet can effectively train much deeper networks than was previously possible.

The standard implementations of ResNet include variants with different depths, typically ranging from 18 layers to 152 layers. ResNet-50, ResNet-101, and ResNet-152 are among the most popular configurations, with the number indicating the count of layers. These models have shown remarkable success in benchmark tasks such as image classification, detection, and segmentation on challenging datasets like ImageNet and COCO.

In terms of performance, ResNet models have dramatically reduced error rates in classification tasks compared to their predecessors. For instance, ResNet-50 and ResNet-101 have become go-to models for many researchers due to their balance of depth and computational efficiency, providing excellent accuracy without the exponential increase in computational cost usually associated with deeper models.

Moreover, the adaptability of ResNet is one of its standout features. It has been successfully applied in settings beyond image recognition, including audio signal processing and reinforcement learning. The architecture's ability to be modified and extended makes it a versatile tool for a wide range of applications in machine learning.

Overall, the ResNet architecture has proven to be a significant advancement in neural network design, enabling deeper and more powerful networks that can handle the increasing complexity of tasks across various domains. Its ongoing influence in research and application underscores its critical role in the evolution of deep learning technologies.

Training diffusion probabilistic models presents a unique set of challenges that are both computational and theoretical in nature. These models operate by learning to reverse a diffusion process, gradually converting random noise into a structured data distribution over many iterative steps. While the results can be impressive, achieving them is far from straightforward.

One of the most significant hurdles in training diffusion models is their high computational cost. Each training iteration involves simulating the reverse diffusion process across multiple timesteps, which can be computationally intensive especially as the number of timesteps increases. This makes training such models resource-heavy, requiring powerful hardware and often resulting in long training times. Researchers have explored methods such as subsampling timesteps during training to reduce computational overhead. This approach is discussed in "Improved Denoising Diffusion Probabilistic Models" by Alex Nichol and Prafulla Dhariwal [2], where they introduce techniques to selectively skip timesteps without compromising the model's performance.

The performance of diffusion models heavily relies on the precise tuning of hyperparameters, including the learning rate, the number of diffusion steps, and the noise schedule. Determining the optimal settings for these parameters can be a trial-and-error process that requires extensive experimentation and computational resources.

Although diffusion models are generally more stable during training compared to other generative models like GANs, they are not completely free from stability issues. Problems can arise from numerical instabilities during the simulation of the reverse diffusion process, particularly when dealing with very low probability events in high-dimensional spaces.

Diffusion models are often applied to high-dimensional data such as images, which can complicate the training process. The curse of dimensionality can exacerbate the challenges in modeling the data distribution accurately, requiring more sophisticated techniques and deeper networks. Techniques such as principal component analysis (PCA) and autoencoders have been used to reduce the dimensionality of data before training diffusion models. This can simplify the learning process and improve the manageability of high-dimensional data.

Measuring the performance of diffusion models can also be challenging. Traditional metrics like log-likelihood are not always informative for image quality in visual tasks. Alternatives like Inception Score or Fréchet Inception Distance are commonly used, but they have their limitations and may not fully capture the model's capabilities in generating diverse and realistic images.

A practical challenge in training diffusion models for image generation is balancing the level of detail in the generated images with the diversity of the output. Overtraining on specific features can lead to loss of diversity (mode collapse), while too much randomness can reduce the realism of the generated images.

Despite these challenges, diffusion models are continuously being refined and improved, making them increasingly viable for a range of applications. Advances in computational resources, algorithmic optimizations, and better understanding of the underlying processes are helping to mitigate these difficulties, broadening the practical use of diffusion models in the field of generative modeling.

Exploring Stochastic Differential Equations (SDEs) as a modeling approach for the diffusion process has become a significant area of research in the development of diffusion probabilistic models. SDEs are used to provide a mathematical framework for modeling the continuous dynamics of the random variables involved in the diffusion process. This approach has several advantages, particularly in terms of flexibility, precision, and the ability to model complex stochastic behaviors.

Stochastic Differential Equations are used to model systems that exhibit random behavior and are influenced by continuous stochastic noise. In the context of diffusion models, SDEs describe how an image (or another data type) gradually transitions from a meaningful configuration to a state of pure noise—and vice versa—over continuous time. This is fundamentally different from the discrete step-wise approach seen in earlier diffusion models, where the process was modeled as a sequence of discrete steps.

In diffusion probabilistic models, the forward process is modeled as adding noise over time until the data becomes indistinguishable from Gaussian noise. The reverse process, which is of primary interest, involves learning the reverse of this noise-adding process. By employing SDEs, researchers can model this reverse process more naturally and fluidly as it provides a continuous-time framework, which can capture the nuances of the noise reduction at any point in time.

SDEs allow the diffusion process to be defined in continuous time, giving a finer control over the dynamics of the process. This can lead to more precise and controlled generation of the data, as the model can potentially make adjustments at any moment in the continuous timeline. The use of SDEs enables the modeling of more complex dynamics that cannot be captured by simple discrete transitions. This is particularly useful in applications where the data exhibits intricate stochastic behaviors. It is also possible to implement more efficient sampling algorithms that leverage the continuous nature of the process. For instance, adaptive step-size solvers can be used to speed up the reverse diffusion process without loss of quality.

The seminal work by Yang Song et al. in their paper "Score-Based Generative Modeling through Stochastic Differential Equations" published at ICLR 2021 [3], is a notable contribution to this area, and the correspondence between the forward and reverse-time processes that it formalises is illustrated in Figure 2.2. They propose a framework that uses score-based methods and SDEs to generate samples. By defining the generative model as a solution to an SDE, they are able to leverage the properties of differential equations to model the diffusion process more robustly and efficiently.

Despite the advantages, implementing SDEs in diffusion models also introduces new challenges. The primary issue is the computational cost associated with solving SDEs, especially in high-dimensional spaces typical of image data. Additionally, designing and training models to solve these equations requires sophisticated techniques and a deep understanding of both the

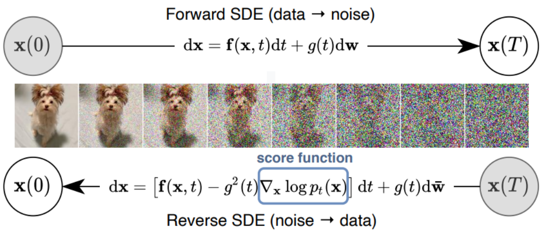


Figure 2.2: Solving a reverse time SDE yields a score-based generative model. Transforming data to a simple noise distribution can be accomplished with a continuous-time SDE. This SDE can be reversed if we know the score of the distribution at each intermediate time step [3]

theoretical and practical aspects of stochastic calculus.

In conclusion, the exploration of SDEs in diffusion probabilistic models represents an exciting advancement, offering a more detailed and continuous approach to modeling the diffusion process. This method holds promise for improving the quality and efficiency of generative models, expanding their applicability in various complex and nuanced tasks.

2.2 Analysis of Competing Products

Denoising Diffusion Probabilistic Models (DDPMs), Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs) are all prominent types of generative models, each with unique strengths and applications.

2.2.1 Variational Autoencoders (VAE)

Variational Autoencoders (VAEs) have been a significant part of the generative modeling landscape since their introduction by Kingma and Welling in their groundbreaking paper "Auto-Encoding Variational Bayes" in 2013 [4]. VAEs are especially valued for their robustness in learning latent representations and their ability to generate new data points with variations that are meaningful in a learned latent space.

VAEs are particularly celebrated for their ability to learn well-structured latent spaces. This means that similar data points result in close points in the latent space, making these models excellent for tasks that involve data interpolation, anomaly detection, and more. This has been critical for advancements in fields like bioinformatics where understanding the underlying structure of data is crucial.

Early criticisms of VAEs focused on their tendency to produce blurrier samples compared to GANs. However, advancements have been made to address this issue. Techniques such as using hierarchical latent variables, improving the variational bound, or incorporating better decoder architectures have significantly enhanced the quality of the generated samples. For instance, the introduction of VQ-VAE (Vector Quantized VAE) by van den Oord et al. [5] and its successor VQ-VAE-2 by Razavi et al. [6] showcases substantial improvements in the fidelity and diversity of generated images.

VAEs have found applications in a variety of fields beyond just image generation. In drug discovery, VAEs are used to generate novel molecular structures by learning from known chemical compounds. Similarly, in finance, they help in modeling complex financial instruments and in risk management by understanding the distributions of various financial parameters. These applications benefit from VAEs' ability to model complex, high-dimensional data in a compressed, interpretable format.

The flexibility of VAEs allows them to be effectively combined with other neural network architectures, enhancing their utility and performance. For example, the integration of VAEs with Recurrent Neural Networks (RNNs) has led to improved models for sequence data, which is useful in natural language processing and sequential image analysis.

The theoretical aspects of VAEs have also been extensively explored, leading to a deeper understanding and better implementations. Innovations such as the β-VAE, introduced by Higgins et al. [7], offer a way to balance latent channel capacity and independence constraints to control the disentanglement of latent representations, enhancing both interpretability and control over generative models.

2.2.2 Generative Adversarial Networks (GAN)

Generative Adversarial Networks (GANs) have revolutionized the field of generative modeling since their introduction by Ian Goodfellow and his colleagues in 2014 [8]. The unique adversarial training approach of GANs, involving a duel between a generator and a discriminator, has led to substantial advancements in generating high-fidelity and realistic images. The achievements of GANs span various domains, from art creation to medical image analysis.

One of the most significant achievements of GANs is their ability to produce incredibly high-quality and realistic images. For example, the development of models like BigGAN [9] and StyleGAN has pushed the boundaries of how lifelike generated images can be. StyleGAN, in particular, introduced by Karras et al. [10], has become famous for its ability to manipulate and control different aspects of the image generation process, leading to stunningly detailed and controllable outputs. These models demonstrate a leap in the visual quality of generated images, achieving unprecedented resolution and realism.

GANs have been successfully applied to a wide range of applications. In the field of fashion, GANs are used for creating new clothing designs and for virtual try-ons. In video games and virtual reality, they help generate realistic environments and characters. Additionally, GANs have made significant impacts in the medical field, where they are used for data augmentation in medical imaging and for generating synthetic patient data that can be used for training other AI models without privacy concerns.

GANs have also been pivotal in advancing unsupervised and semi-supervised learning techniques. The ability of GANs to learn complex distributions without extensive labeled data has made them valuable for scenarios where labeled data is scarce or expensive to obtain. For instance, models like the Semi-Supervised GAN (SGAN) [11] leverage this capability to improve learning efficiency and accuracy using smaller amounts of labeled data supplemented with larger amounts of unlabeled data.

Despite their potential, GANs were initially known for being difficult to train, with issues such as mode collapse and non-convergence. Over the years, researchers have proposed various techniques to stabilize GAN training, such as introducing new architectures, loss functions, and regularization techniques. The introduction of the Wasserstein GAN (WGAN) [12] marks a significant development in this area, addressing the issue of training stability through a novel loss function that provides a smoother gradient.

The theoretical understanding of GANs has also evolved, leading to more robust and efficient models. Research has delved into the intricacies of GAN dynamics, exploring how different components of the network interact during training. This theoretical exploration has not only improved the practical implementations of GANs but has also contributed to the broader understanding of neural networks and deep learning.

2.3 Specification of Software Standards

- **Google Python Style Guide** is a comprehensive set of guidelines that Google provides to maintain a consistent and high-quality codebase across its Python projects.
- **NumPy** is a fundamental library for scientific computing in Python, providing support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays.
- **PyTorch** is an open-source machine learning library developed by Facebook, known for its flexibility and ease of use in building and training deep learning models, particularly popular for research and prototyping.

2.4 ResNet Background Theory

Residual Networks represent a seminal advancement in deep learning that emerged to address the vanishing gradient problem occurring in very deep neural networks. ResNet fundamentally altered approaches to network architecture design, enabling the training of networks that are substantially deeper than those previously feasible. This development has had profound implications for the field of computer vision and beyond.

Before ResNet, increasing network depth to improve performance had reached a bottleneck. Typically, deeper networks begin to suffer from the vanishing gradient problem—where gradients become too small to make meaningful adjustments in weights during backpropagation—leading to saturation and then rapid degradation of training accuracy. Moreover, deeper networks often encounter a complexity that makes them computationally expensive and harder to optimize.

ResNet introduced a novel solution to these issues through the use of residual learning. The central hypothesis was that if deeper networks are approximations of shallower ones, then it should be easier to optimize the residual mappings rather than the desired underlying mapping.

The core idea behind ResNet is the introduction of the residual block, which includes shortcut connections (also known as skip connections). These connections allow the input to a layer to be added to its output, effectively forming an identity mapping that skips one or more layers. Mathematically, if $H(x)$ is an underlying mapping to be learned by a few stacked layers, and x is the input, then these layers in ResNet fit another mapping of (2.1). The original function thus becomes (2.2).

$$F(x) := H(x) - x \tag{2.1}$$

$$H(x) = F(x) + x \tag{2.2}$$

This setup makes it easier to optimize the network because learning the residual mapping $F(x)$ is simpler than learning the unreferenced mapping $H(x)$. In practice, this means that even very deep networks can be trained directly through standard backpropagation. This approach addresses both the degradation problem—ensuring that adding more layers does not lead to higher training error—and reduces the impact of the vanishing gradients by providing an alternative pathway for gradient flow during backpropagation.

The standard ResNet models are implemented with varying depths, including ResNet-18, ResNet-34, ResNet-50, ResNet-101, and ResNet-152, which indicate the number of layers. Each ResNet block is a composition of several convolutional layers, batch normalization layers, and ReLU activations. In deeper ResNet versions (50, 101, 152), bottleneck layers are introduced

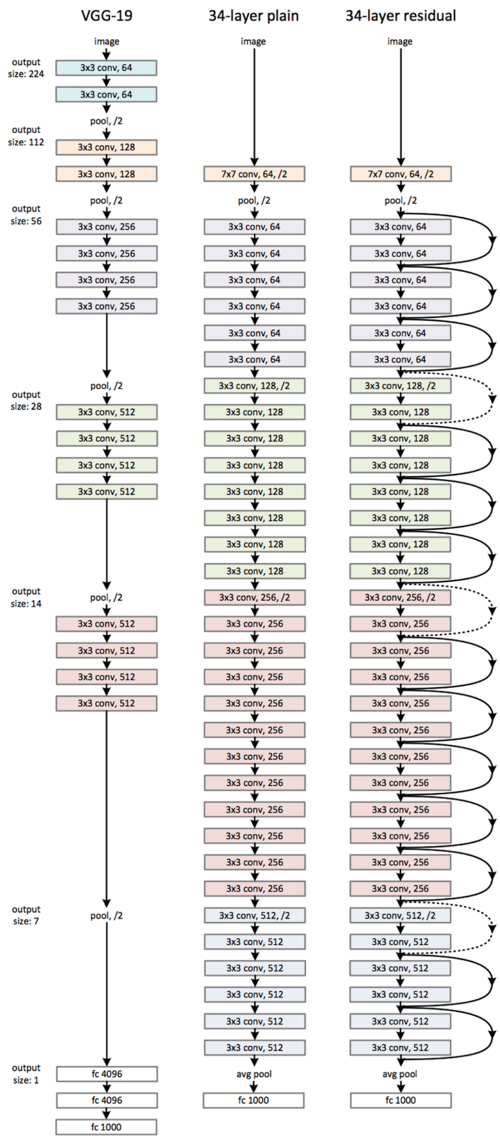


Figure 2.3: Example network architectures for ImageNet. **Left**: the VGG-19 model [13] (19.6 billion FLOPs) as a reference. **Middle**: a plain network with 34 parameter layers (3.6 billion FLOPs). **Right**: a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions. [1]

to reduce the model size and complexity. Each convolutional layer within these bottlenecks is tightly packed with 1×1 and 3×3 convolutions.

2.4.1 Plain Network

The plain network architecture referenced in the context of ResNet typically comprises stacked convolutional layers, where each layer is followed by batch normalization and a ReLU activation function (Figure 2.3, **Left**). This architecture builds on the basic principles of CNNs, which are designed to capture hierarchical patterns in image data through a series of filtering and pooling operations.

In a plain network, convolutional layers form the core components. These layers apply a set of learnable filters to the input image, capturing various features at different levels of abstraction. For example, the initial layers might capture simple features like edges and textures, while deeper layers capture more complex patterns such as object parts.

Following each convolutional operation, batch normalization is applied. This technique normalizes the input layer by adjusting and scaling the activations, helping to maintain mean output close to 0 and the output standard deviation close to 1. This normalization helps to combat issues in training deep networks by stabilizing the learning process and reducing the number of training epochs required.

After batch normalization, a non-linear activation function—typically the Rectified Linear Unit (ReLU)—is used. The ReLU function introduces non-linear properties to the system, allowing the network to learn more complex patterns. It is defined as $f(x) = max(0, x)$, which has been shown to accelerate the convergence of stochastic gradient descent compared to traditional sigmoid or tanh functions.

The architecture often includes pooling layers, which reduce the spatial size of the representation, diminishing the amount of parameters and computation in the network. Pooling helps in making the detection of features invariant to scale and orientation changes.

Towards the end of the network, fully connected layers are typically used to map the learned features to the desired output size. For instance, in classification tasks, these layers might culminate in a softmax activation function that distributes the output into probability scores corresponding to various classes.

The plain architecture, while foundational, encounters significant challenges as network depth increases:

- **Vanishing/Exploding Gradients**: In very deep networks, gradients propagated back through the network can vanish (become very small) or explode (become very large), which significantly hampers the ability to learn.

- **Degradation Problem**: As depth increases, network accuracy gets saturated and then degrades rapidly. Interestingly, this degradation is not due to overfitting, and adding more layers leads to higher training error.

The plain network architecture is crucial for understanding the baseline upon which ResNet improves. By addressing the shortcomings of the plain networks, especially the degradation problem through the introduction of skip connections (or shortcut connections), ResNet allows much deeper networks to be trained effectively. This capability to train deeper networks without a drop in performance has set new benchmarks in various machine learning tasks and underscores the transformative impact of ResNet in the landscape of deep learning.

2.4.2 Residual Network

The cornerstone of the ResNet architecture is the concept of residual learning. As networks deepen, training becomes increasingly difficult due to the vanishing gradient problem, where gradients of the network's layers diminish as they propagate back through the network during training. Traditional networks attempt to learn the desired underlying mapping directly. In contrast, ResNet reformulates the layers as learning residual functions with reference to the layer inputs, thereby simplifying the learning process (Figure 2.3, **Right**).

A typical ResNet is composed of several stacked residual blocks. Each residual block has two main pathways:

- **Identity Shortcut Connection**: This pathway allows the input to bypass the main layers of the block and connect directly to the output of the block, adding the input x to the output of the block's layers $F(x)$. This direct path forwards the input without any alteration, preserving the strength of the gradient as it back-propagates through the network.
- **Weighted Layers**: Typically consisting of two or three convolutional layers (depending on the variant of ResNet), each followed by batch normalization and a ReLU activation function. These layers aim to learn the residual functions, denoted as $F(x)$, where x is the input to the layers.

The output of a residual block is then (2.2), where $F(x)$ represents the learned residual function and x is the identity mapping. The addition of $F(x)$ and x is element-wise and may involve dimensionality adjustments (using 1×1 convolutions) if necessary to match the dimensions.

ResNet comes in various depths, which are generally tailored to the complexity of the tasks they are intended to perform. Common variants include:

- **ResNet-18** and **ResNet-34**: These versions use blocks with two convolutional layers each.
- **ResNet-50**, **ResNet-101**, and **ResNet-152**: These deeper models utilize bottleneck blocks designed to reduce the model size and computational load. A bottleneck block contains three layers: a 1×1 convolution that reduces the dimension, a 3×3 convolution, and another 1×1 convolution that restores the dimension.

In this project the denoising network is not one of the standard ImageNet classifiers but a residual UNet (ResUNet) assembled from the basic two-layer residual block described above. Bottleneck blocks of the kind used in ResNet-50 are designed for the deep, heavily downsampled feature hierarchies demanded by 224×224 ImageNet classification; at the 32×32 resolution of CIFAR-10 the spatial extent is exhausted after only a few downsampling stages, so the additional 1×1 compression they provide yields little benefit for a considerable increase in implementation complexity. A shallower network built from basic blocks is therefore adopted, and the classification head is replaced by an encoder-decoder structure so that the network maps an image to an image rather than to a class score, as a denoising network must.

The resulting architecture retains two downsampling stages at the 32×32 resolution of CIFAR-10. At the 64×64 resolution of CelebA a third stage is inserted and the channel widths are narrowed accordingly, so that the two configurations remain comparable in size.

- **Encoder stage 1** maps the three input channels to 64 feature channels at 32×32, after which max pooling halves the resolution.
- **Encoder stage 2** maps 64 channels to 128 at 16×16, followed by a second max pooling to 8×8.

- A **residual bridge** operates on the 128 channels at the coarsest resolution.
- **Decoder stage 2** upsamples to 16×16 by transposed convolution and concatenates the stage-2 encoder features, giving 256 channels.
- **Decoder stage 1** upsamples to 32×32 and concatenates the stage-1 encoder features, giving 128 channels.
- A final 1×1 convolution projects the 128 channels back to the three image channels.

Each residual block follows the two-layer form used in ResNet-18 and ResNet-34: a 3×3 convolution, a normalisation layer, a non-linear activation, a second 3×3 convolution and a further normalisation, with the block input added to the result. Group normalisation is used in place of the batch normalisation of the original ResNet. The running statistics that batch normalisation accumulates would be estimated over clean images, whereas the input actually presented to a denoising network is the noised image x_t, whose distribution changes with the timestep; group normalisation avoids this mismatch by normalising within each example. Because the channel count is preserved within a block, the shortcut is a pure identity mapping and no 1×1 projection is required. The connections between corresponding encoder and decoder stages are concatenations rather than additions, which is the convention inherited from the UNet family and is what allows the fine spatial detail lost during downsampling to be recovered in the decoder.

2.4.3 Reference Denoising Architecture

The network proposed above is assessed against the denoising architecture introduced with the DDPM itself [14], which serves as the reference design throughout this work. It is also a UNet built from residual blocks, but differs from the network of the previous section in four respects.

- It uses **four resolution stages** rather than two or three, so the coarsest feature map is reached through a deeper hierarchy.
- **Self-attention** is inserted at the 16×16 resolution and at the bottleneck, giving each spatial position access to the whole feature map rather than only to its convolutional neighbourhood.
- A **skip connection is taken from every residual block**, not one per resolution stage, so the decoder receives a considerably finer record of the encoder activations.
- Resolution is changed by **strided convolution and nearest-neighbour upsampling followed by convolution**, in place of max pooling and transposed convolution.

Because the channel count varies within a stage, the shortcut in these blocks cannot be a pure identity mapping and a 1×1 projection is applied where the dimensions differ. The configuration published for CIFAR-10 uses a base width of 128 channels and amounts to roughly 35 million parameters, which is more than an order of magnitude larger than the network proposed here. Comparing the two at their published sizes would confound architecture with capacity, so the base width is reduced until the parameter counts agree to within a few per cent; the comparison then reflects the arrangement of the layers rather than the number of them.

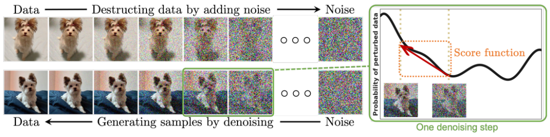


Figure 2.4: Diffusion models smoothly perturb data by adding noise, then reverse this process to generate new data from noise. Each denoising step in the reverse process typically requires estimating the score function (see the illustrative figure on the right), which is a gradient pointing to the directions of data with higher likelihood and less noise. [15]

2.5 Diffusion Model Background Theory

Diffusion models (Figure 2.4) operate by modeling the data generation process as a Markov chain of gradual, stepwise transformations from a data distribution into a predetermined noise distribution, typically Gaussian noise. This process is characterized mathematically by a series of forward diffusion steps that progressively add noise to the data until only noise remains.

2.5.1 Forward Process

The forward process (or the diffusion process) can be described by the following stochastic differential equation (SDE):

$$dX_t = -\frac{1}{2}\beta(t)X_t dt + \sqrt{\beta(t)}dW_t \tag{2.3}$$

Here, X_t represents the data at time t, $\beta(t)$ is a variance schedule that governs the noise level at each step, and dW_t denotes the increment of a Wiener process, reflecting the stochastic nature of the noise addition. The parameter $\beta(t)$ typically increases with t, indicating that the amount of noise added increases over time.

2.5.2 Reverse Process

The reverse process aims to reconstruct the original data from the noise. This is where diffusion models utilize the concept of denoising. In the reverse process, the model learns to estimate the gradients of the log probability density of the data with respect to the data itself, given the noisy data at each step. This gradient is used to perform a step-by-step denoising through the following approximate reverse SDE:

$$dX_t = \left[-\frac{1}{2}\beta(t)X_t - \beta(t)\nabla_{X_t}\log p_t(X_t)\right]dt + \sqrt{\beta(t)}\mathrm{d}\bar{W}_t \tag{2.4}$$

The term $\nabla_{X_t}\log p_t(X_t)$ represents the score function that the model learns; this function guides the reverse diffusion process to effectively recover the data distribution from the noise.

2.5.3 Scheduling

The forward process in a diffusion model gradually adds noise to the original data over a series of steps. This is often done in a controlled manner according to a predefined noise schedule. The noise schedule determines the amount of noise added at each step and is typically parameterized by a variance schedule $\beta(t)$, where t denotes the timestep.

The reverse process involves denoising the data. This process uses the learned parameters to gradually remove the noise from the data, aiming to revert to the original data distribution. The reverse process follows a reverse noise schedule, which ideally mirrors the forward noise schedule but in reverse order. The model learns to predict the clean data from the noisy data step-by-step, effectively learning the conditional distribution of the cleaner data given the noisier data.

The optimization of the noise schedule is a key area of research in diffusion models. The schedule needs to be carefully designed to balance the trade-offs between the model's stability, sampling speed, and quality of generated samples. Researchers often experiment with different forms of the noise schedule, such as linear, cosine, or learned schedules, each providing different benefits:

- **Linear Schedules** increment noise linearly over time and are straightforward but may not always capture the nuances of more complex data distributions.
- **Cosine Schedules** vary noise according to a cosine function, often leading to smoother transitions in noise levels.
- **Learned Schedules** involve learning the noise schedule directly from the data, allowing the model to adaptively determine the best way to add and remove noise based on the specific characteristics of the data.

In this project, we will explore and compare various scheduling methods to determine the optimal noise schedule for the Denoising Diffusion Probabilistic Model (DDPM). The investigation will involve systematically implementing these different scheduling strategies to assess their impact on the model's performance. By analyzing the effects of these schedules on the fidelity, stability, and efficiency of the generated outputs, the study aims to identify the most effective scheduling method tailored to the specific characteristics and requirements of the DDPM. This comparative analysis will not only enhance our understanding of the model's dynamics but also contribute to optimizing its generative capabilities in practical applications.

3 Results

3.1 Experimental Configuration

The six runs specified in Section 1.1.2 were carried out on a single NVIDIA GeForce RTX 4070. Every run used $T = 1000$ diffusion steps under the cosine schedule, a batch size of 128, the Adam optimiser at a learning rate of 2×10^{-4}, thirty epochs, and random horizontal flipping as the only augmentation. Every configuration was trained three times, under seeds 0, 1 and 2; the seed governs weight initialisation, batch order and augmentation, and within a given seed all three architectures receive the same one. Figures below are reported as the mean over the three runs with the sample standard deviation, and comparisons between two architectures are paired within seed. Training used bfloat16 mixed precision with the channels-last memory format; because these settings alter neither the model definition nor the objective, they affect the wall-clock cost of a run but not the comparison between architectures.

Table 3.1 records the resulting parameter budgets. The proposed network and the plain variant are identical in size, as an identity shortcut introduces no parameters, so the comparison between them isolates residual learning as the only variable. The reference network was narrowed until it fell within ten per cent of the proposed network; it is in fact the smallest of the three, which means that any advantage it shows cannot be attributed to additional capacity.

Table 3.1: Parameter counts. The proposed and plain networks are identical in size by construction; the reference network is the smallest of the three.

Dataset	Architecture	Parameters	Relative to ResUNet
CIFAR-10	ResUNet	2 473 475	—
CIFAR-10	plain UNet	2 473 475	100.0%
CIFAR-10	DDPM UNet	2 356 739	95.3%
CelebA	ResUNet	2 612 419	—
CelebA	plain UNet	2 612 419	100.0%
CelebA	DDPM UNet	2 356 739	90.2%

3.2 Training Convergence

Table 3.2 reports the epoch-averaged noise-prediction loss at five points during training. Two patterns are visible, and they point in opposite directions.

The first is an early advantage for residual learning. After a single epoch the proposed network has the lowest loss on both datasets in every one of the three runs, ahead of the plain variant by 5.4% on average on CIFAR-10 and 11.1% on CelebA. Since the two networks are identical apart from the identity shortcut, this difference is attributable to that shortcut alone, and it is consistent with the argument of the original ResNet work that a residual formulation is easier to optimise than an unreferenced one.

The second pattern is that the advantage does not survive. Somewhere between the third and the sixth epoch, depending on the seed, the plain network overtakes the proposed one on both datasets, and at convergence it remains marginally ahead in every run, by 0.0003 on both datasets. The consistency is striking: the sign never changes across the six paired comparisons, and a paired test over the three seeds gives $t(2) = 6.1$ on CIFAR-10 and 9.0 on CelebA. The

Table 3.2: Epoch-averaged training loss. Lower is better. The proposed network leads at the first epoch on both datasets and is overtaken by the fifth.

		Epoch				
Dataset	Architecture	1	5	10	20	30
CIFAR-10	ResUNet	**0.13775**	0.07111	0.06619	0.06249	0.06108
CIFAR-10	plain UNet	0.14633	**0.07033**	**0.06555**	**0.06212**	0.06084
CIFAR-10	DDPM UNet	0.18508	0.07135	0.06593	0.06165	**0.05967**
CelebA	ResUNet	**0.14205**	0.04735	0.04110	0.03844	0.03716
CelebA	plain UNet	0.16413	0.04705	0.04055	0.03799	0.03679
CelebA	DDPM UNet	0.15588	**0.04316**	**0.03779**	**0.03573**	**0.03476**

magnitude is not: 0.0003 is one half of one per cent of the loss. Section 3.4 shows that this near-equality on the training objective is profoundly misleading about the quality of the images the two networks actually generate.

The reference network behaves differently again. It begins worst on CIFAR-10, at 0.18508, which is what a deeper network with more layers to coordinate would be expected to do, but it finishes with the lowest loss on both datasets under every seed. The epoch at which it takes the lead is itself unstable, falling anywhere between the second and the twelfth on CIFAR-10 and between the first and the second on CelebA. The full per-epoch record is given in Appendix A.1.

3.3 Restoration Accuracy

The restoration diagnostics corrupt a held-out test image to a chosen timestep and measure how completely the network recovers it in a single step. Table 3.3 reports both measures over a thousand test images at six noise levels.

Three observations follow. The first is that at $t = 50$ the three networks are indistinguishable to four decimal places on SSIM and to within 0.04 dB on PSNR. An image at that noise level is barely corrupted, and almost any denoiser recovers it; the measurement discriminates only as the corruption grows.

The second is that the proposed and plain networks cannot be separated. Their mean SSIM differs by 0.0012 on CIFAR-10 and 0.0011 on CelebA, and their mean PSNR by 0.01 dB and 0.02 dB. Repeating the whole procedure under three seeds puts those figures in context. The seed-to-seed standard deviation of mean SSIM is 0.0013 for the proposed network and 0.0030 for the plain one on CIFAR-10, which is of the same order as the 0.0031 that separates them. The paired differences do favour the proposed network consistently, at $t(2) = 2.0$ on CIFAR-10 and 2.4 on CelebA, but they run in the opposite direction to the training loss, and PSNR changes sign between seeds altogether. The restoration measures are individually stable and jointly inconclusive: they can be trusted to a thousandth, and at that resolution they do not agree on which network is better.

The third is that the reference network leads at every noise level on both datasets and on both measures, and its margin grows with t. Its advantage is consistently larger on CelebA than on CIFAR-10, which is the pattern the higher resolution was introduced to expose.

Table 3.3: Restoration accuracy against noise level. SSIM (higher better) above, PSNR in decibels (higher better) below. Differences between the proposed and plain networks are negligible throughout; the reference network leads at every level.

	CIFAR-10			CelebA		
t	ResUNet	plain	DDPM UNet	ResUNet	plain	DDPM UNet
SSIM						
50	0.9712	0.9713	**0.9713**	0.9708	0.9710	**0.9727**
100	0.9388	0.9393	**0.9395**	0.9444	0.9451	**0.9485**
200	0.8701	0.8707	**0.8740**	0.8947	0.8953	**0.9027**
400	0.7286	0.7281	**0.7382**	0.7996	0.8004	**0.8153**
600	0.5681	0.5694	**0.5826**	0.6891	0.6894	**0.7154**
800	0.3529	0.3438	**0.3642**	0.5117	0.5023	**0.5510**
Mean	0.7383	0.7371	**0.7450**	0.8017	0.8006	**0.8176**
PSNR (dB)						
50	33.50	33.52	**33.54**	35.51	35.54	**35.77**
100	29.89	29.91	**29.95**	32.20	32.24	**32.49**
200	26.21	26.23	**26.34**	28.80	28.83	**29.10**
400	22.35	22.37	**22.52**	25.07	25.10	**25.40**
600	19.61	19.61	**19.80**	22.23	22.25	**22.61**
800	16.63	16.50	**16.87**	18.82	18.74	**19.18**
Mean	24.70	24.69	**24.84**	27.10	27.12	**27.43**

3.4 Distribution-Level Quality

The measures reported so far compare the networks as denoisers. Table 3.4 compares them as generators, using two thousand samples drawn from each model through the complete thousand-step reverse process.

Table 3.4: Fréchet Inception Distance from 2048 samples per model, for each of the three seeds, with the mean and sample standard deviation. Lower is better.

Dataset	Architecture	Seed 0	Seed 1	Seed 2	Mean $\pm$ SD
CIFAR-10	ResUNet	94.93	128.23	89.03	104.07 $\pm$ 21.13
CIFAR-10	plain UNet	84.29	145.12	93.15	107.52 $\pm$ 32.86
CIFAR-10	DDPM UNet	68.09	68.66	81.88	**72.88** $\pm$ **7.80**
CelebA	ResUNet	76.06	109.79	63.40	83.08 $\pm$ 23.98
CelebA	plain UNet	135.18	131.47	172.53	146.39 $\pm$ 22.71
CelebA	DDPM UNet	76.93	73.91	93.87	**81.57** $\pm$ **10.76**

This table carries the most important result of the project, and the three seeds are what make it readable. The seed-to-seed standard deviation of FID runs from 7.8 to 32.9, an order of magnitude larger than the variation of any restoration measure, so a comparison drawn from a single run of this length cannot be relied upon. Two of the three comparisons available here dissolve under repetition; one does not.

On CIFAR-10 the proposed and plain networks cannot be separated. Their means are 104.07 and 107.52, the paired difference is -3.46 against a standard deviation of 13.78, and the sign changes between seeds: the proposed network is worse by 10.64 under seed 0, better by 16.89

under seed 1 and better by 4.12 under seed 2. A paired test gives $t(2) = -0.43$. At two resolution stages the identity shortcut makes no difference to generative quality that this experiment can detect.

On CelebA it does. The proposed network is ahead in all three runs, by 59.12, 21.68 and 109.13, for a mean advantage of 63.31 FID. That gap is larger than the seed-to-seed standard deviation of either network, 23.98 and 22.71, and the direction never changes. The evidence is not conclusive at this sample size, a paired test giving $t(2) = -2.50$, which does not reach significance with two degrees of freedom, but a consistent advantage of that size across three independent runs is not readily explained as noise.

The reference architecture leads on CIFAR-10 by 31.19 FID over the proposed network, in every seed, and is also the most stable of the three, with a standard deviation of 7.80 against 21.13 and 32.86. On CelebA it is matched: the mean difference to the proposed network is 1.51 FID with a standard deviation of 33.24, and the sign is mixed. Its Inception Score on CIFAR-10, 5.655 against 4.711 and 4.625, agrees with its FID lead.

Read together with the previous section, the two sets of measurements describe an awkward situation. The restoration measures are stable to a thousandth but cannot separate the proposed and plain networks, and where they do lean they contradict one another. FID separates them decisively on CelebA but varies by tens of points between seeds. **The measurement that is reliable is uninformative, and the measurement that is informative is noisy.** The reason is the same in both cases: the training objective and the restoration diagnostics both average the noise-prediction error over timesteps, whereas generation composes a thousand such predictions in sequence, and errors that are small on average but systematic at particular timesteps compound through that composition. A network can be an excellent denoiser and a poor generator, and only a distribution-level measure sees it.

The difference between the two datasets invites an explanation in terms of depth. At 32×32 the proposed network has two resolution stages and the shortcut changes nothing; at 64×64 it has three and the shortcut is worth sixty FID points. Residual connections exist to make deeper networks optimisable, so a benefit that appears only in the deeper configuration is what the original argument would predict. The present experiment cannot establish this, because depth and resolution vary together; separating them would require the two-stage and three-stage networks to be trained on the same images.

3.5 Memorisation

Because the CelebA samples are recognisable faces, it is necessary to establish that the models are generating rather than reproducing. Table 3.5 reports the mean distance from each generated sample to its nearest training image, against two controls.

Table 3.5: Mean pixel-space L_2 distance to the nearest training image. A generated sample that is no closer than a blurred real image gives no evidence of reproduction.

	CIFAR-10			CelebA		
	ResUNet	plain	DDPM	ResUNet	plain	DDPM
Generated	18.00	16.93	17.06	43.94	47.02	43.56
Real held-out images		18.68			38.41	
Blurred held-out		16.09			35.97	

On CelebA every model produces samples that are *further* from the training set than genuine unseen photographs are, by between thirteen and twenty-two per cent, which settles the question directly.

CIFAR-10 requires the second control. The generated samples sit slightly closer to the training set than real held-out images do, at between 0.91 and 0.96 of the reference distance, and taken alone that could be read as mild reproduction. Blurring a real held-out image with a 3×3 mean filter, however, reduces its distance to 16.09, or 0.86 of the reference. Smoothing an image moves it towards the mean of the data and therefore towards everything in it. All three models remain further away than that purely smoothing-induced floor, so the shortfall is explained by the softness of samples from an undertrained model rather than by copying.

Figure 3.1 shows the eight closest generated and training pairs for the proposed network on CelebA. The nearest training image is a different person in every case.

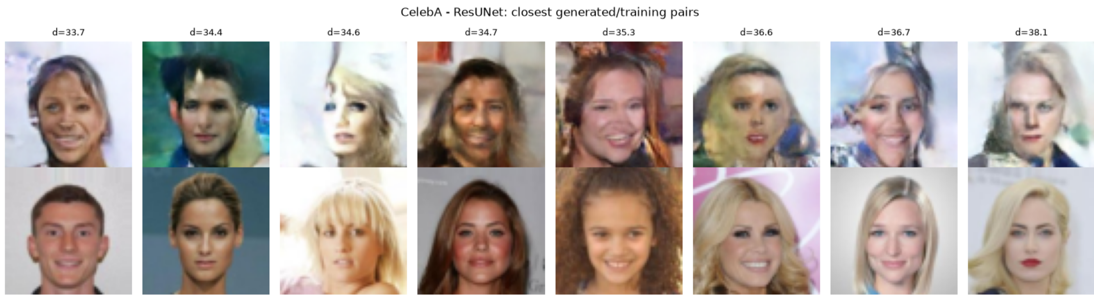


Figure 3.1: The eight CelebA samples closest to the training set (top) with their nearest training image (bottom), for the proposed network. No pair depicts the same person.

3.6 Computational Cost

Table 3.6 records the steady-state time for one training epoch. The proposed and plain networks are within two per cent of one another on both datasets, confirming in practice what the parameter counts assert in principle: removing an identity shortcut changes neither the parameter count nor the arithmetic performed.

Table 3.6: Steady-state training time per epoch. The first epoch of each dataset absorbs convolution autotuning and is excluded.

Dataset	ResUNet	plain UNet	DDPM UNet
CIFAR-10 (32×32)	16.0 s	15.8 s	38.5 s
CelebA (64×64)	32.0 s	31.4 s	68.3 s

The reference network costs 2.4 times as much per epoch on CIFAR-10 and 2.2 times as much on CelebA, despite having fewer parameters than either competitor. The cost is incurred by depth and by attention rather than by weight count. Its advantage on CIFAR-10 is decisive and worth the price; on CelebA it is matched by the proposed network at less than half the training cost.

3.7 Generated Samples

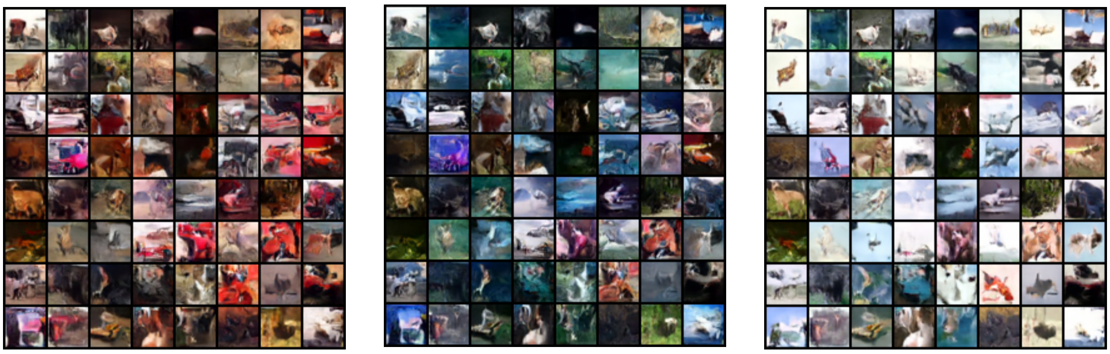
3.7.1 A Note on the Reverse Process

The first attempt at unconditional sampling produced saturated noise from every model. The cause was not the models but the implementation of the reverse step. Writing the posterior mean directly as $\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta\right) / \sqrt{\alpha_t}$ is algebraically correct only while the implied estimate of x_0 remains inside the data range. The factor $1/\sqrt{\alpha_t}$ reaches 31.6 at $t = 999$ under the clipped cosine schedule, so any error in ε_θ is amplified by that factor and accumulates over the thousand steps of the reverse process. Tracing the sample statistics confirmed the effect: the standard deviation of x grew from 1.0 at the start of the trajectory to approximately 16 by its end, and over 95% of pixels finished outside the valid range. The same divergence occurred in single precision, which excludes reduced precision as the cause.

The remedy is the one used in the reference implementation: recover x_0 explicitly, clip it to $[-1, 1]$, and rebuild the posterior mean from the clipped estimate. With this correction the terminal standard deviation falls to between 0.48 and 0.64 across the six models, matching the scale of the training data, and no pixel leaves the valid range. All samples reported here use the corrected procedure. The episode illustrates a property of the reverse process that the forward derivation does not make obvious: the algebraic form of the posterior mean is stable only for a network that is already accurate, and the clipping step is what makes sampling robust to a network that is not.

3.7.2 Samples

Figure 3.2 and Figure 3.3 show sixty-four samples drawn from each model. Every model was given the same initial noise, so differences between the panels are attributable to the denoising network.



ResUNet plain UNet DDPM UNet

Figure 3.2: Unconditional CIFAR-10 samples after thirty epochs, all three models starting from identical noise.

On CIFAR-10 all three models produce coherent local texture and plausible figure-ground separation, and some samples suggest recognisable categories such as animals or vehicles, but none is identifiable with confidence. This is the expected outcome for networks of two and a half million parameters trained for eleven thousand optimisation steps; the published CIFAR-10 configuration is roughly fourteen times larger and is trained for two orders of magnitude longer.

The CelebA panels are where the FID result becomes visible. All three models generate face-like images with correct global arrangement, but the plain network's faces are noticeably less coherent, with more frequent failures of facial structure, which is consistent with its FID of 135.18 against 76.06 and 76.93 for the other two. The proposed and reference networks are difficult to separate by eye, as their almost identical scores would suggest. Colour saturation is exaggerated in all three, a known consequence of clipping the estimate of x_0 at every step of an undertrained reverse process.

ResUNet plain UNet DDPM UNet

Figure 3.3: Unconditional CelebA samples at 64 × 64 after thirty epochs, all three models starting from identical noise.

3.7.3 Why the Samples Are Indistinct

The samples above are recognisable in kind but not in particular, and it is worth separating the two reasons for that, because only one of them is a property of the method.

The first is simply the scale of the experiment. Table 3.7 sets the configuration used here beside the one published with the original DDPM [14]. The networks trained for this work are roughly fourteen times smaller and receive sixty-eight times fewer optimisation steps, which places the total training computation about three orders of magnitude below the reference. The resulting Fréchet Inception Distance is about twenty times worse. Nothing in that comparison is surprising, and it is the reason Section 3.4 treats only the relative ordering of the three networks as meaningful.

Table 3.7: Training budget compared with the published CIFAR-10 configuration. The difference in computation is approximately three orders of magnitude.

	This work	Ho et al.	Ratio
Parameters	2.47 M	35.7 M	14×
Optimisation steps	11 700	800 000	68×
Relative computation	1	$\approx 10^3$	$\approx 10^3$
CIFAR-10 FID	68.09	≈ 3.17	21×

The second reason explains why the failure takes the particular form of indistinctness rather than, say, visible artefacts. The training objective is a squared error, and when a network is uncertain the prediction that minimises squared error is the average of the outcomes it considers possible. Averaging over several plausible completions of a region produces a smooth one. This is the same mechanism that gives Variational Autoencoders their characteristic softness, discussed in Section 2.2.1, and it is not specific to either architecture tested here.

What distinguishes a diffusion model is that it does not have to commit to a whole image at once. Dividing the task into a thousand steps keeps the uncertainty at each step small, and the noise added back at every step, in the term $\sigma_t z$ of the reverse process, pushes the trajectory towards one specific outcome instead of the average of many. Both mitigations depend on each step being accurate. An undertrained network makes a small error at every one of the thousand steps, the averaging reasserts itself, and the softness returns. The indistinctness observed here is therefore a symptom of the training budget rather than of the formulation, and the remedies set out in the Conclusions are addressed to that budget.

Figure 3.4 shows the reverse trajectory of the proposed network on both datasets, sampled every twenty steps. Structure emerges late: the first two thirds of the trajectory refine what remains essentially noise, and the recognisable image forms over the final few hundred steps.

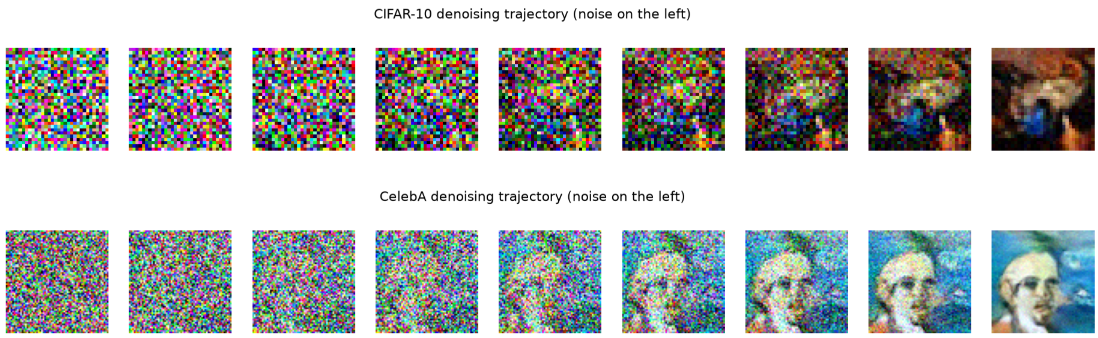


Figure 3.4: Reverse trajectories of the proposed network on CIFAR-10 (top) and CelebA (bottom), with pure noise on the left and the final sample on the right.

3.8 Summary

Figure 3.5 collects the training curves and the restoration measurements. Four findings follow from the experiment.

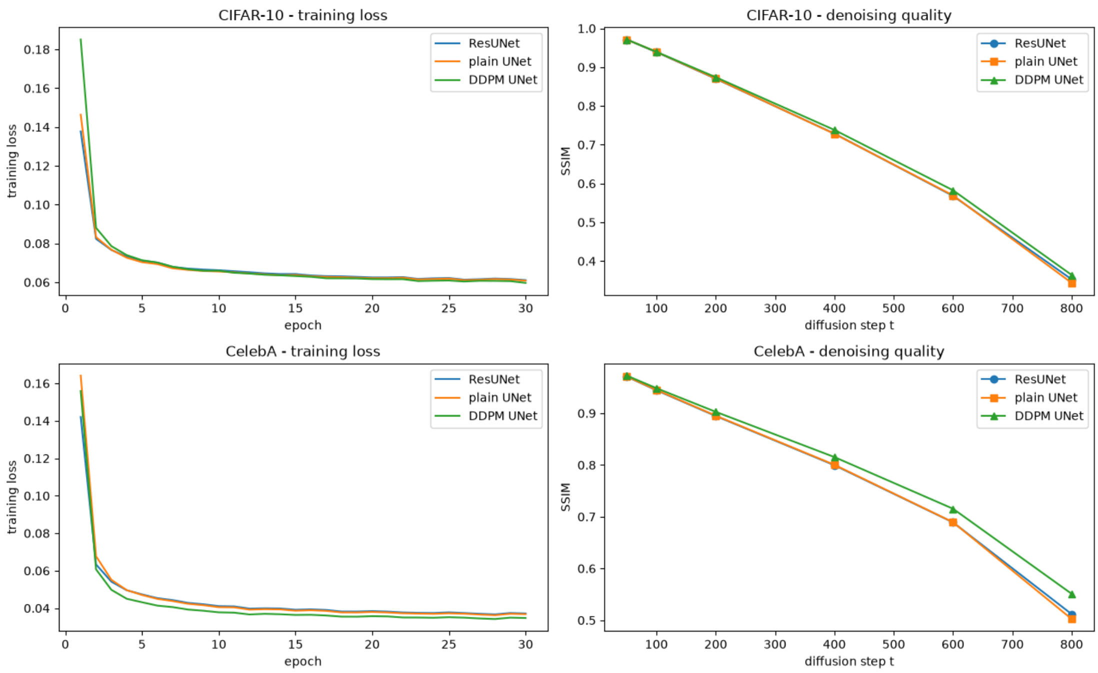


Figure 3.5: Training loss (left) and denoising accuracy against noise level (right) for CIFAR-10 (top) and CelebA (bottom).

- **Residual shortcuts accelerate early optimisation.** The proposed network reaches a lower loss than its shortcut-free twin after a single epoch in all six paired runs, by 5.4% on average on CIFAR-10 and 11.1% on CelebA. As the two networks are otherwise identical, the shortcut is the only possible cause.

- **Restoration measures cannot separate the two networks.** Their final training loss, mean SSIM and mean PSNR agree to within half of one per cent, and the two that lean at all lean in opposite directions: the loss favours the plain network in every run, SSIM favours the proposed one, and PSNR changes sign between seeds.
- **Generative quality separates them on the deeper configuration only.** At 64×64, where the proposed network has three resolution stages, it leads in all three seeds by a mean of 63.31 FID. At 32×32, with two stages, the sign changes between seeds and the paired difference is -3.46 ± 13.78. Networks that denoise identically well do not generate equally well, and no restoration measure predicted it.
- **The reference architecture remains the strongest overall.** It leads every restoration measure on both datasets and CIFAR-10 FID by 31.19 in every seed, using fewer parameters than either competitor, at roughly 2.3 times the training cost. It is also the most stable, with an FID standard deviation of 7.80 against 21.13 and 32.86. Only on CelebA FID is it matched, by the proposed network at less than half the cost.

The hypothesis stated in Chapter 1 therefore receives partial and conditional support. The combination of residual learning with a UNet backbone does not improve the quality of generated images in general; at two resolution stages it makes no difference this experiment can detect. At three stages, where the network is deep enough for the optimisation difficulty that residual learning was invented to address, the proposed network leads in every seed by a margin larger than the seed-to-seed variation, and that advantage is invisible to every measure except the distribution-level one. Three seeds are not enough to place a confidence interval on the claim, but they are enough to show that the two-stage result of a single run was noise and the three-stage result was not.

Conclusions

This project set out to determine whether combining residual learning with the UNet backbone conventionally used as the denoising network of a diffusion model improves the quality of the images it generates. A Denoising Diffusion Probabilistic Model was implemented from first principles and the diffusion framework was held fixed while three denoising networks were substituted into it: the proposed residual UNet, an otherwise identical network with the identity shortcut removed, and the reference architecture of Ho et al. narrowed to a matching parameter budget. Each was trained on CIFAR-10 at 32×32 and on CelebA at 64×64 under identical conditions.

Findings

The answer to the question as posed is that residual learning helps, but conditionally, and not in the way the project anticipated.

The shortcut makes optimisation easier at the outset. After one epoch the proposed network leads its shortcut-free twin in all six paired runs, by 5.4% on average on CIFAR-10 and 11.1% on CelebA, and since the two differ in nothing else the shortcut is the only possible cause. That advantage is gone by the fifth epoch and does not reappear: at convergence the restoration measures cannot separate the two networks, and the two that lean at all lean in opposite directions, the training loss favouring the plain network and structural similarity the proposed one, both by less than half of one per cent.

They are not, however, indistinguishable as generators, and repeating each configuration under three seeds is what makes the distinction legible. On CelebA the proposed network leads in every seed, by 59, 22 and 109 Fréchet Inception Distance, for a mean advantage of 63.31 against a seed-to-seed standard deviation of about 23 for either network. On CIFAR-10 the same comparison yields -3.46 ± 13.78 with the sign changing between seeds, which is to say no effect at all; the apparent reversal seen in a single run was noise. The difference between the two datasets tracks depth, the CelebA configuration having three resolution stages against two for CIFAR-10, and residual connections exist precisely to make deeper networks optimisable. Where the network is shallow enough not to need one, adding it changes nothing; where it is deep enough to struggle without one, it is worth sixty points of FID.

The reference architecture remains the strongest network overall. It leads on every restoration measure on both datasets and on CIFAR-10 FID by a wide margin, while using fewer parameters than either competitor, at roughly 2.3 times the training cost per epoch. Only on CelebA FID is it matched, and there the proposed network achieves the same quality for less than half the compute.

Two results of a different kind emerged from the work and are worth recording. The first is methodological: a network can denoise as well as another and generate images that are 78% worse by FID. The training objective averages the noise-prediction error uniformly over timesteps, whereas generation composes a thousand such predictions in sequence, so errors that are small on average but systematic at particular timesteps compound through that composition. Evaluating a diffusion model by reconstruction alone is not sufficient. The second is practical: the algebraic form of the posterior mean used in the reverse process is numerically stable only for a network whose noise prediction is already accurate. Without clipping the implied estimate of x_0 at every step, the reverse process diverged from a standard deviation of 1 to approximately 16, in single precision as well as mixed, and produced saturated noise from every model.

Limitations

Each configuration was trained three times. Three runs are enough to establish that a sign is consistent and to estimate a standard deviation, but not to support a formal claim of significance: the paired test on the CelebA result gives $t(2) = -2.50$, which does not reach the conventional threshold with two degrees of freedom. The finding rests on the direction being the same in every run and the magnitude exceeding the observed spread, not on a p-value. The depth explanation offered for the difference between datasets is consistent with the evidence but not established by it, since depth and resolution vary together here; separating them would require the two-stage and three-stage configurations to be trained on the same images.

The seed study also showed that the Fréchet Inception Distance is far noisier at this scale than the restoration measures, with standard deviations between 7.8 and 32.9 against thousandths for SSIM. Any comparison of diffusion models drawn from a single training run of this length, including several reported in the earlier sections of this work before the repetition was carried out, should be treated with corresponding caution.

Thirty epochs amounts to roughly eleven thousand optimisation steps, some two orders of magnitude short of the schedule used for the published CIFAR-10 configuration, and the networks are about fourteen times smaller. The absolute FID values are correspondingly poor and should not be compared with the literature; only the relative ordering between the three networks, which were trained identically, carries meaning. The Fréchet Inception Distance was computed from two thousand samples, which is the smallest number at which the estimate is not degenerate given the dimensionality of the Inception feature space, and remains biased upward at that size.

One standard practice was omitted. Diffusion implementations customarily sample not from the trained weights themselves but from an exponential moving average of them, which suppresses the fluctuation of the final iterates and typically improves the Fréchet Inception Distance appreciably. No such average was maintained here, so the absolute sample quality reported is lower than the same training run would otherwise have produced. The omission applies identically to all six runs and so does not affect the comparison between them.

The memorisation check operates in pixel space and would not detect a reproduction that had been shifted or recoloured. Negative Log-Likelihood was excluded on the grounds that this implementation fixes the reverse-process variances rather than learning them. The comparison against Variational Autoencoders and Generative Adversarial Networks set out in Chapter 2 was not carried out.

Further work

The immediate priority is more repetition. Three seeds established which of the single-run findings survive, but the surviving one is supported by consistency of sign rather than by a significance test; ten seeds would permit a confidence interval, and at the cost of the deterministic sampler described below would remain affordable.

The depth hypothesis is directly testable. Training two-stage, three-stage and four-stage variants of the proposed network on a single dataset, each with and without the shortcut, would isolate depth from resolution and determine whether the benefit of residual learning grows with the former as the argument predicts.

A separate line of work concerns the absolute quality of the samples rather than the comparison between architectures. Three changes would address it, in descending order of expected effect and ascending order of cost. Maintaining an exponential moving average of the weights and sampling from it is the cheapest, requiring a few lines and no change to the experimental design. Extending training by an order of magnitude is the most certain, since Section 3.7.3 attributes the indistinctness principally to the budget. Replacing the ancestral sampler with the deterministic variant of Song et al. would allow comparable quality from roughly fifty reverse steps instead of a thousand, which would also reduce the cost of computing the Fréchet Inception Distance by a factor of twenty and so make the repeated runs recommended above considerably cheaper. None of the three alters the architectures under test, so each could be applied to all six configurations without disturbing the comparison.

Beyond that, learning the reverse-process variances rather than fixing them would make the variational bound informative and bring Negative Log-Likelihood within reach, and the Variational Autoencoder and Generative Adversarial Network baselines would place the diffusion results in the wider context that Chapter 2 describes but does not measure.

A Appendices

A.1 Per-Epoch Training Loss

Table 3.2 in Chapter 3 reports the training loss at five checkpoints. Table A.2 gives the complete record for the six runs under seed 0, so that the shape of the curves discussed in Section 3.2 can be inspected directly. The other two seeds follow the same trajectory and are not reproduced; their final values appear in Table 3.4 and the underlying record is available in the accompanying repository.

The epoch at which one network overtakes another is not itself stable across seeds, and Table A.1 records where it falls. The proposed network is overtaken by its shortcut-free variant somewhere between the third and the sixth epoch on both datasets, which is consistent with the early advantage being real but short-lived. The reference network is more erratic: on CIFAR-10 it takes the lead as early as the second epoch under one seed and as late as the twelfth under another, although it finishes ahead in every case. These figures are a further reminder that quantities read from a single run should not be quoted to a precision the experiment does not support.

Table A.1: The epoch at which one network first attains a lower training loss than another.

Comparison	Dataset	Seed 0	Seed 1	Seed 2
plain overtakes ResUNet	CIFAR-10	4	4	3
plain overtakes ResUNet	CelebA	5	3	6
DDPM UNet leads both	CIFAR-10	11	2	12
DDPM UNet leads both	CelebA	2	1	2

A.2 Network Specifications

The comparison in this work rests on the claim that the proposed and plain networks are identical in size and that the reference network is comparable. Tables A.3 and A.4 give the breakdown from which that claim can be checked.

The proposed and shortcut-free networks share every table entry, because an identity shortcut introduces no parameters. Only the presence of the addition $x+h$ at the end of each block differs, which is why the two rows of Table 3.1 are equal to the digit.

Two features of Table A.3 are worth noting. The first is that the coarsest decoder dominates the budget, holding 52% of the parameters on CIFAR-10 and 49% on CelebA, because concatenation doubles the channel count before the block that follows it. The second is that adding a third resolution stage for CelebA increases the total by only 5.6%: the channel widths were narrowed from $(64, 128)$ to $(32, 64, 128)$ so that the deeper network would remain comparable in size to the shallower one, which is what permits the two datasets to be discussed on the same footing.

The reference network reaches a similar total by a different distribution. Its upward path carries 53% of the parameters, because a skip connection is taken from every residual block rather than once per resolution stage, so each of the twelve upward blocks receives a concatenated input. The attention layers account for only 3.6% of the weights while contributing a large share of the runtime, which is why this network costs roughly 2.3 times as much per epoch despite being the smallest of the three.

Table A.2: Epoch-averaged noise-prediction loss for the six runs under seed 0. Lower is better.

	CIFAR-10			CelebA		
Epoch	ResUNet	plain	DDPM UNet	ResUNet	plain	DDPM UNet
1	0.13775	0.14633	0.18508	0.14205	0.16413	0.15588
2	0.08237	0.08327	0.08805	0.06335	0.06774	0.06071
3	0.07665	0.07676	0.07861	0.05415	0.05513	0.04980
4	0.07322	0.07265	0.07399	0.04957	0.04963	0.04503
5	0.07111	0.07033	0.07135	0.04735	0.04705	0.04316
6	0.07017	0.06936	0.07014	0.04535	0.04496	0.04136
7	0.06790	0.06719	0.06799	0.04430	0.04379	0.04057
8	0.06705	0.06637	0.06672	0.04283	0.04234	0.03927
9	0.06655	0.06582	0.06591	0.04207	0.04159	0.03863
10	0.06619	0.06555	0.06593	0.04110	0.04055	0.03779
11	0.06570	0.06511	0.06491	0.04094	0.04044	0.03761
12	0.06517	0.06455	0.06448	0.03977	0.03924	0.03665
13	0.06458	0.06409	0.06386	0.03994	0.03948	0.03699
14	0.06424	0.06365	0.06360	0.03984	0.03935	0.03677
15	0.06424	0.06373	0.06322	0.03915	0.03866	0.03639
16	0.06352	0.06308	0.06281	0.03938	0.03891	0.03647
17	0.06315	0.06275	0.06206	0.03904	0.03855	0.03610
18	0.06305	0.06264	0.06202	0.03822	0.03775	0.03549
19	0.06278	0.06236	0.06196	0.03821	0.03773	0.03546
20	0.06249	0.06212	0.06165	0.03844	0.03799	0.03573
21	0.06250	0.06213	0.06158	0.03818	0.03772	0.03561
22	0.06265	0.06235	0.06160	0.03771	0.03727	0.03505
23	0.06170	0.06142	0.06062	0.03749	0.03712	0.03502
24	0.06204	0.06165	0.06080	0.03741	0.03700	0.03489
25	0.06218	0.06187	0.06091	0.03775	0.03726	0.03519
26	0.06132	0.06108	0.06040	0.03740	0.03704	0.03495
27	0.06156	0.06128	0.06075	0.03700	0.03659	0.03452
28	0.06190	0.06161	0.06071	0.03668	0.03626	0.03422
29	0.06165	0.06140	0.06059	0.03741	0.03700	0.03496
30	0.06108	0.06084	0.05967	0.03716	0.03679	0.03476

Table A.3: Residual UNet, by stage. The figures apply unchanged to the shortcut-free variant. Each decoder concatenates the encoder features of the corresponding stage, so its block operates on the sum of the upsampled and skipped channel counts.

Stage	Channels	Resolution	Parameters
CIFAR-10, two resolution stages			
Timestep embedding MLP	—	—	33 024
Encoder 1	$3 \to 64$	32×32	84 160
Encoder 2	$64 \to 128$	16×16	386 048
Bridge	128	8×8	312 192
Decoder 2	$\to 256$	16×16	1 279 872
Decoder 1	$\to 128$	32×32	377 792
Final 1×1 convolution	$\to 3$	32×32	387
Total			*2 473 475*
CelebA, three resolution stages			
Timestep embedding MLP	—	—	33 024
Encoder 1	$3 \to 32$	64×64	23 648
Encoder 2	$32 \to 64$	32×32	100 864
Encoder 3	$64 \to 128$	16×16	386 048
Bridge	128	8×8	312 192
Decoder 3	$\to 256$	16×16	1 279 872
Decoder 2	$\to 128$	32×32	377 792
Decoder 1	$\to 64$	64×64	98 784
Final 1×1 convolution	$\to 3$	64×64	195
Total			*2 612 419*

Table A.4: Reference DDPM UNet, by functional group. The same configuration is used for both datasets, since none of its parameter counts depends on the input resolution.

Group	Parameters
Timestep embedding MLP	131 712
Input convolution	896
Downward residual blocks (8)	523 328
Downward attention	33 536
Downsampling convolutions (3)	83 104
Middle block (residual, attention, residual)	181 504
Upward residual blocks (12)	1 240 640
Upward attention	50 304
Upsampling convolutions (3)	110 784
Output normalisation and convolution	931
Total	*2 356 739*

A.3 Memorisation Check: Remaining Configurations

Figure 3.1 in Chapter 3 shows the closest generated and training pairs for the proposed network on CelebA. The remaining five configurations are given here, and support the same conclusion: in no case is a generated sample a reproduction of a training image.

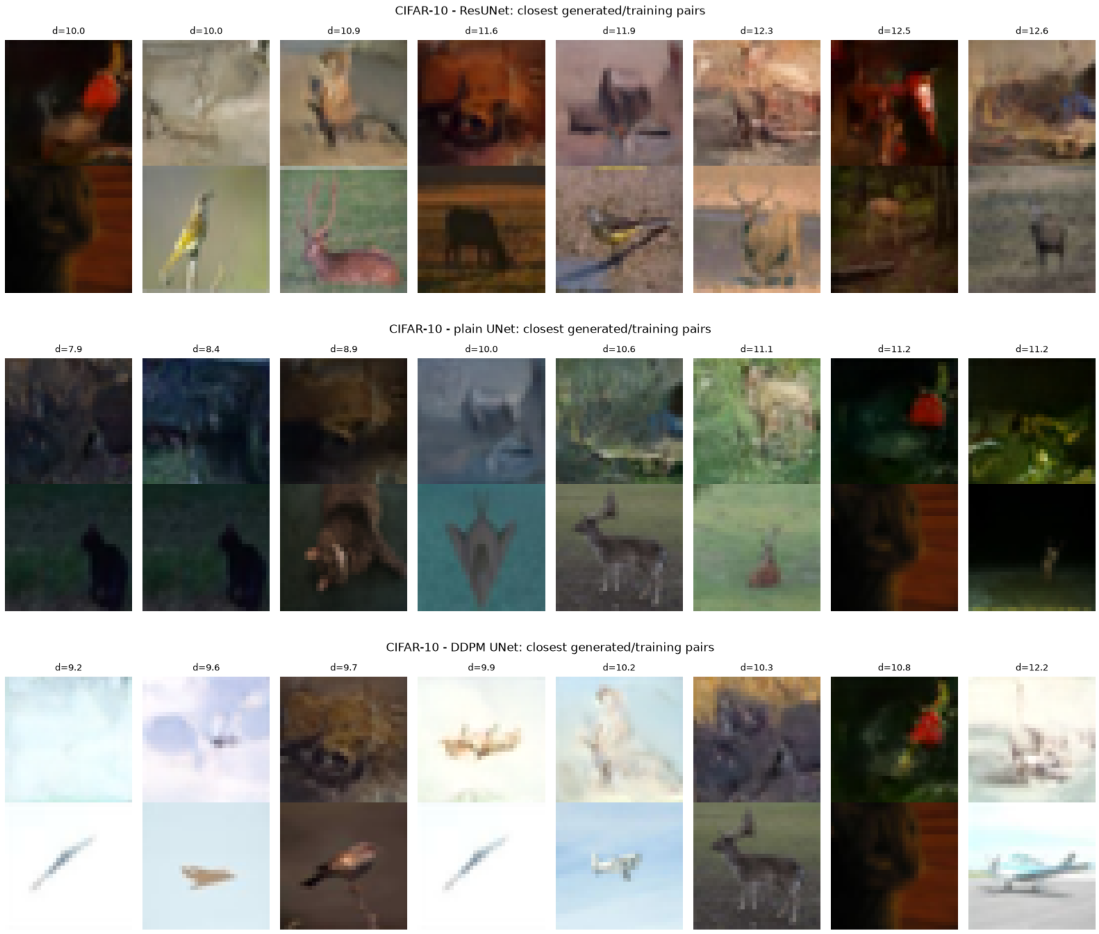


Figure A.1: CIFAR-10: the eight samples closest to the training set (top row of each pair) with their nearest training image (bottom row), for the proposed, shortcut-free and reference networks in turn.

A.4 Computing Environment

All eighteen runs were carried out on the machine described in Table A.5. Six configurations trained for thirty epochs under each of three seeds amount to approximately five hours of training, and the evaluation adds a further two and a half, dominated by drawing the two thousand samples per model required for the Fréchet Inception Distance. The complete experiment therefore occupies about seven and a half hours on a single consumer graphics card, which is what makes the repetition recommended in the Conclusions practical rather than aspirational.

Training used bfloat16 mixed precision with the channels-last memory format, and TF32 matrix multiplication with the cuDNN autotuner enabled. These settings affect only the arithmetic used to evaluate a fixed model, not the model itself, and were applied identically to all six runs.

The implementation is a single Jupyter notebook containing the network definitions, the dif-

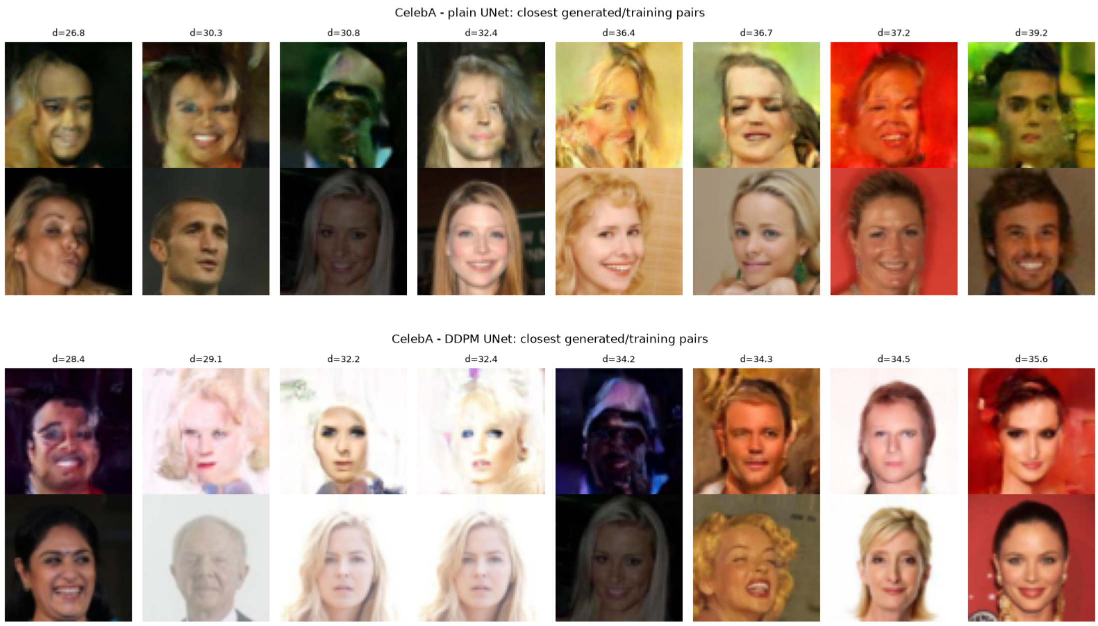


Figure A.2: CelebA: the shortcut-free and reference networks. No pair depicts the same person.

Table A.5: Hardware and software used for every run reported in this work.

Component	Version
GPU	NVIDIA GeForce RTX 4070, 12 GB
Driver	591.86
Operating system	Windows 11
Python	3.12.10
PyTorch	2.14.0+cu130
torchvision	0.29.0+cu130
CUDA runtime	13.0
cuDNN	9.24.0
torchmetrics	1.9.0
torch-fidelity	0.4.0
scikit-image	0.26.0
NumPy	2.5.2

fusion process, the training loop and every evaluation reported here. It includes a self-check routine that verifies the noise schedule is monotone and converges to the standard normal distribution, that the closed-form forward process is consistent, that the network output depends on the timestep, that the sampling path executes, and that the parameter budgets of the three architectures satisfy the constraints stated in Section 1.1.2. Datasets are decoded once into unsigned eight-bit tensors and cached, so that only the first execution pays the preprocessing cost, and training is skipped for any architecture whose checkpoint already records the full number of epochs.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2016. arXiv: 1512.03385 [cs.CV].

[2] A. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," 2021. arXiv: 2102.09672 [cs.LG].

[3] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," 2020. arXiv: 2011.13456 [cs.LG].

[4] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," 2013. arXiv: 1312.6114 [stat.ML].

[5] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural discrete representation learning," 2017. arXiv: 1711.00937 [cs.LG].

[6] A. Razavi, A. van den Oord, and O. Vinyals, "Generating diverse high-fidelity images with VQ-VAE-2," 2019. arXiv: 1906.00446 [cs.LG].

[7] I. Higgins et al., "Beta-VAE: Learning basic visual concepts with a constrained variational framework," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=Sy2fzU9gl

[8] I. J. Goodfellow et al., "Generative adversarial networks," 2014. arXiv: 1406.2661 [stat.ML].

[9] A. Brock, J. Donahue, and K. Simonyan, "Large scale GAN training for high fidelity natural image synthesis," 2018. arXiv: 1809.11096 [cs.LG].

[10] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," 2018. arXiv: 1812.04948 [cs.NE].

[11] A. Odena, "Semi-supervised learning with generative adversarial networks," 2016. arXiv: 1606.01583 [stat.ML].

[12] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," 2017. arXiv: 1701.07875 [stat.ML].

[13] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. arXiv: 1409.1556 [cs.CV].

[14] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," 2020. arXiv: 2006.11239 [cs.LG].

[15] L. Yang et al., "Diffusion models: A comprehensive survey of methods and applications," 2024. arXiv: 2209.00796 [cs.LG].